Orbital illumination, opacity, and atmospheric heating

Radiative forcing connects three calculations with different responsibilities. The orbit sets the sunlight incident on each column. Gas and cloud opacity describe how material absorbs and scatters that light and thermal radiation. A radiation state combines these inputs, solves vertical transfer, and converts flux divergence into the energy source used by the dynamical integrator.

The following three sections follow that order. They share a single set of units and shape conventions and culminate in a small six-face Uranus example. The models are deliberately simple reference implementations; the grey coefficients and prescribed cloud abundances are demonstration inputs, not a calibrated Uranus climate model.

1. Orbital illumination

Scope and interface

The first step is the radiation arriving at the top of the atmosphere. `paddle/orbital/orbital.py` implements `OrbitalForcing`, a PyTorch module that supplies this illumination geometry. Its small stage-forcing method also applies the heating calculated later in 3. Radiation state and the Uranus example. `paddle/orbital/planets.py` provides `Earth`, `Jupiter`, `Saturn`, `Uranus`, and `Neptune` subclasses. These are fixed elliptical orbits for atmospheric experiments, with spherical surfaces and constant spin. They do not calculate atmospheric absorption or solve radiative transfer.

All dimensional constructor arguments use SI: luminosity in W, semi-major axis in m, period and time in s, and rotation rate in rad/s. Angles are radians. Surface longitude is east-positive and latitude is planetocentric. Longitude and latitude are floating PyTorch tensors with broadcastable shapes on the same device. Time is a single Python float in seconds. For a time series, call `insolation` once per time, as the figure-generation script does.

Kepler iterations, substellar coordinates, and inverse-square irradiance use Python scalar math on the CPU. Only the substellar coordinates are converted to tensors on the grid device; grid-sized incidence and masking run there. This follows the atmospheric time-stepping interface: one time, many columns.

`insolation(lon, lat, time)` returns:

- `mu0`: nonnegative cosine of the stellar zenith angle, zero at night;
- `beam`: normal-to-ray irradiance in W/m^2, masked to zero at night;
- `distance`: star-to-planet distance in m;
- `substellar_lon`: east longitude in the interval [-pi, pi);
- `substellar_lat`: planetocentric latitude in [-pi/2, pi/2].

The first two outputs have the broadcast spatial shape. Distance is a Python float. Substellar longitude and latitude are scalar tensors matching their respective grid coordinates in dtype and device. Horizontal incident flux is `mu0 * beam`.

Kepler's equation

For eccentricity $0 \le e < 1$, period P, and initial mean anomaly M_0, define

$$M(t) = M_0 + \frac{2\pi t}{P}, \qquad M = E - e \sin E.$$

The implementation wraps M to $[-\pi, \pi)$ and solves for E with 32 Newton iterations:

$$E_{k+1} = E_k - \frac{E_k - e \sin E_k - M}{1 - e \cos E_k}.$$

The initial guess is $E_0 = M$ for $e \le 0.8$, otherwise $E_0 = \operatorname{sign}(M)\pi$ (positive at zero). The high-eccentricity start avoids the poor Newton steps near periapsis that occur with $E_0 = M$. The fixed iteration count supports TorchScript; it is not an adaptive error estimate. Extremely near-parabolic cases require separate accuracy assessment. For the supplied planets $e < 0.054$.

Distance and true anomaly follow:

$$r = a(1 - e \cos E), \qquad f = 2 \operatorname{atan2}\left(\sqrt{1+e}\sin(E/2), \sqrt{1-e}\cos(E/2)\right).$$

In particular, periapsis and apoapsis are $a(1-e)$ and $a(1+e)$. Only M advances uniformly: orbital speed increases near periapsis.

J2000 frames and the seasonal phase

For the presets, `time=0` denotes JD 2451545.0 TDB (J2000); time is elapsed TDB seconds, not a UTC timestamp. The orbit is referenced to the **mean J2000 ecliptic and equinox**. Spin poles are given in the **J2000 equatorial** frame. These are distinct frames, connected here by a fixed obliquity $\epsilon_0 = 23.43928^\circ$.

Let I be orbital inclination, Ω ascending-node longitude, and $\omega = \varpi - \Omega$ argument of perihelion. Using active, right-handed rotations, construct perihelion and transverse unit vectors:

$$[\mathbf{P}\ \mathbf{Q}] = R_x(\epsilon_0) R_z(\Omega) R_x(I) R_z(\omega) \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

The heliocentric direction is $\mathbf{P}\cos f + \mathbf{Q}\sin f$. The direction **toward the Sun** has the opposite sign:

$$\mathbf{s} = -\mathbf{P}\cos f - \mathbf{Q}\sin f.$$

For pole right ascension α_p and declination δ_p, define a fixed planet-equatorial basis

$$\mathbf{k} = (\cos\delta_p \cos\alpha_p, \cos\delta_p \sin\alpha_p, \sin\delta_p), \qquad \mathbf{u} = (-\sin\alpha_p, \cos\alpha_p, 0), \qquad \mathbf{v} = \mathbf{k} \times \mathbf{u}.$$

The `solar_basis` tuple of CPU float constants contains the two columns of $-[\mathbf{u}\ \mathbf{v}\ \mathbf{k}]^T[\mathbf{P}\ \mathbf{Q}]$. Multiplication by $(\cos f, \sin f)^T$ gives (s_x, s_y, s_z). With prime-meridian angle $W(t) = W_0 + \dot{W}t$,

$$\delta_\star = \arcsin s_z, \qquad \lambda_\star = \mathrm{wrap}_{[-\pi,\pi)}\left[\operatorname{atan2}(s_y, s_x) - W(t)\right].$$

This construction fixes both the season at J2000 and the longitude origin. Simply specifying an obliquity and setting all mean anomalies to zero would incorrectly synchronize the planets' perihelia and seasons.

For a generic `OrbitalForcing` without a planet preset, the default basis instead gives $(s_x, s_y, s_z) = (\cos f, \cos\varepsilon \sin f, \sin\varepsilon \sin f)$. Thus periapsis coincides with the northern spring equinox by convention. The preset basis replaces this idealized alignment. The basis stays on CPU when `module.to(device)` is called. Like the scalar orbital parameters, it is constructor configuration and is not included in `state_dict`. Reconstruct the same preset/configuration when restoring a model.

Uranus uses the IAU north pole and negative $\dot{W}$. Its `obliquity` attribute is the angle between the **angular momentum** and orbital normal, about 98 degrees. The angle between its IAU north pole and orbital normal is about 82 degrees. Keeping these two conventions distinct preserves the sign of latitude and retrograde rotation. All presets expose east-positive longitude, even where published planetary maps use west-positive longitude.

Incident radiation

At longitude λ and latitude ϕ, the outward surface normal dotted with the stellar direction gives

$$\mu = \sin\phi \sin\delta_\star + \cos\phi \cos\delta_\star \cos(\lambda - \lambda_\star), \qquad \mu_0 = \max(\mu, 0).$$

The inverse-square irradiance and local horizontal flux are

$$S(r) = \frac{L_\star}{4\pi r^2}, \qquad B = S(r)\,\mathbf{1}_{\mu>0}, \qquad F_{\text{horizontal}} = \mu_0 B.$$

A spherical global average is $S/4$. Albedo, atmospheric attenuation, ring shadows, eclipses, refraction, finite stellar angular size, and internal heat flux are outside this module.

The normal beam B and incidence cosine μ_0 are passed separately to the radiation solver: it applies the projection and optical path length itself. Multiplying the beam by μ_0 before passing it would apply the projection twice. The next section supplies the material properties needed to turn this boundary illumination into fluxes through the atmosphere.

Planet presets and provenance

Orbital elements below are the J2000 values of JPL's Table 1 [JPL]. The Earth row represents the Earth-Moon barycenter. The code freezes a, e, I, ϖ, Ω and uses $M_0 = L_0 - \varpi_0$. It sets $P = 360^\circ/(\dot{L} - \dot{\varpi})$ in Julian centuries, each 36525 days; this is the adopted anomalistic period, not an independently tabulated sidereal year. No other secular rates are propagated.

J2000 orbital elements (angles in degrees, axis in au)

Planet	a	e	I	L	varpi	Omega
Earth	1.00000261	0.01671123	-0.00001531	100.46457166	102.93768193	0
Jupiter	5.20288700	0.04838624	1.30439695	34.39644051	14.72847983	100.47390909
Saturn	9.53667594	0.05386179	2.48599187	49.95424423	92.59887831	113.66242448
Uranus	19.18916464	0.04725744	0.77263783	313.23810451	170.95427630	74.01692503
Neptune	30.06992276	0.00859048	1.77004347	-55.12002969	44.96476227	131.78422574

Rotation uses the polynomial constants and linear spin terms in NAIF's `pck00011.tpc` [PCK]. Pole precession and all periodic corrections are omitted, including Neptune's periodic terms at the epoch. Thus these are simplified reference orientations, not exact SPICE IAU frames.

Adopted rotation coefficients

Planet	Pole RA (deg)	Pole Dec (deg)	W0 (deg)	W rate (deg/day)
Earth	0	90	190.147	360.9856235
Jupiter	268.056595	64.495303	284.95	870.5360000
Saturn	40.589	83.537	38.90	810.7939024
Uranus	257.311	-15.175	203.81	-501.1600928
Neptune	299.36	43.46	249.978	541.1397757

All presets use nominal solar luminosity 3.828×10^{26} W [IAU2015] and $1\ \text{au} = 149597870700$ m [IAU2012]. Irradiance at one au is approximately 1361 W/m². Giant-planet spin conventions represent reference rotation, not latitude-dependent atmospheric winds.

JPL's published accuracy estimates for its time-dependent approximation do **not** apply to these frozen-element orbits. Full orbital cycles in the figures illustrate the periodic model, including beyond JPL Table 1's 1800–2050 fitting interval for the outer planets. The simplified Earth spin also omits UT1, polar motion, and nutation. Use a numerical ephemeris and an appropriate orientation model when precise dates, positions, or longitudes are required.

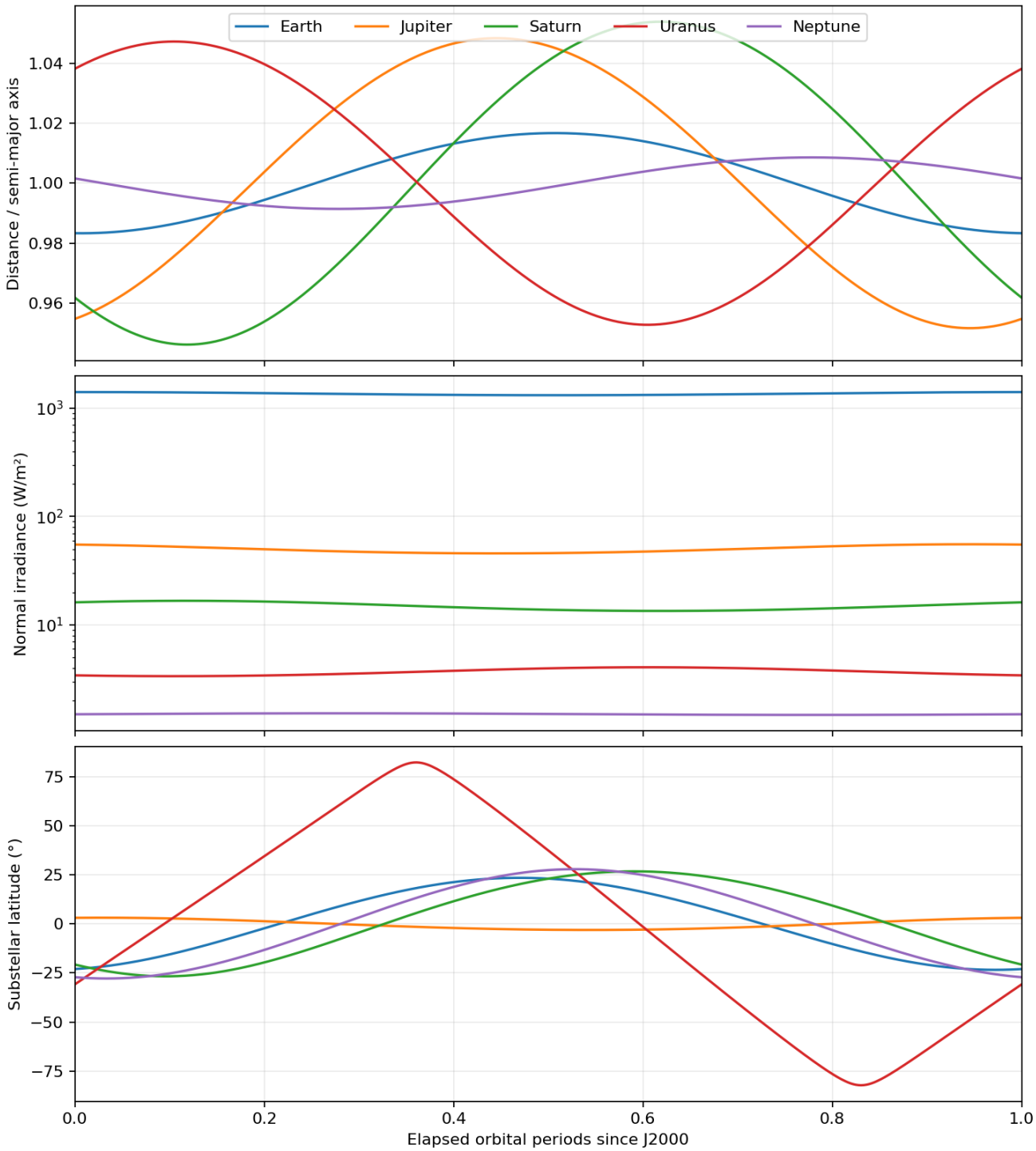
Examples and reproducible figures

```
import torch
from paddle.orbital import Earth, Jupiter, Saturn, Uranus, Neptune

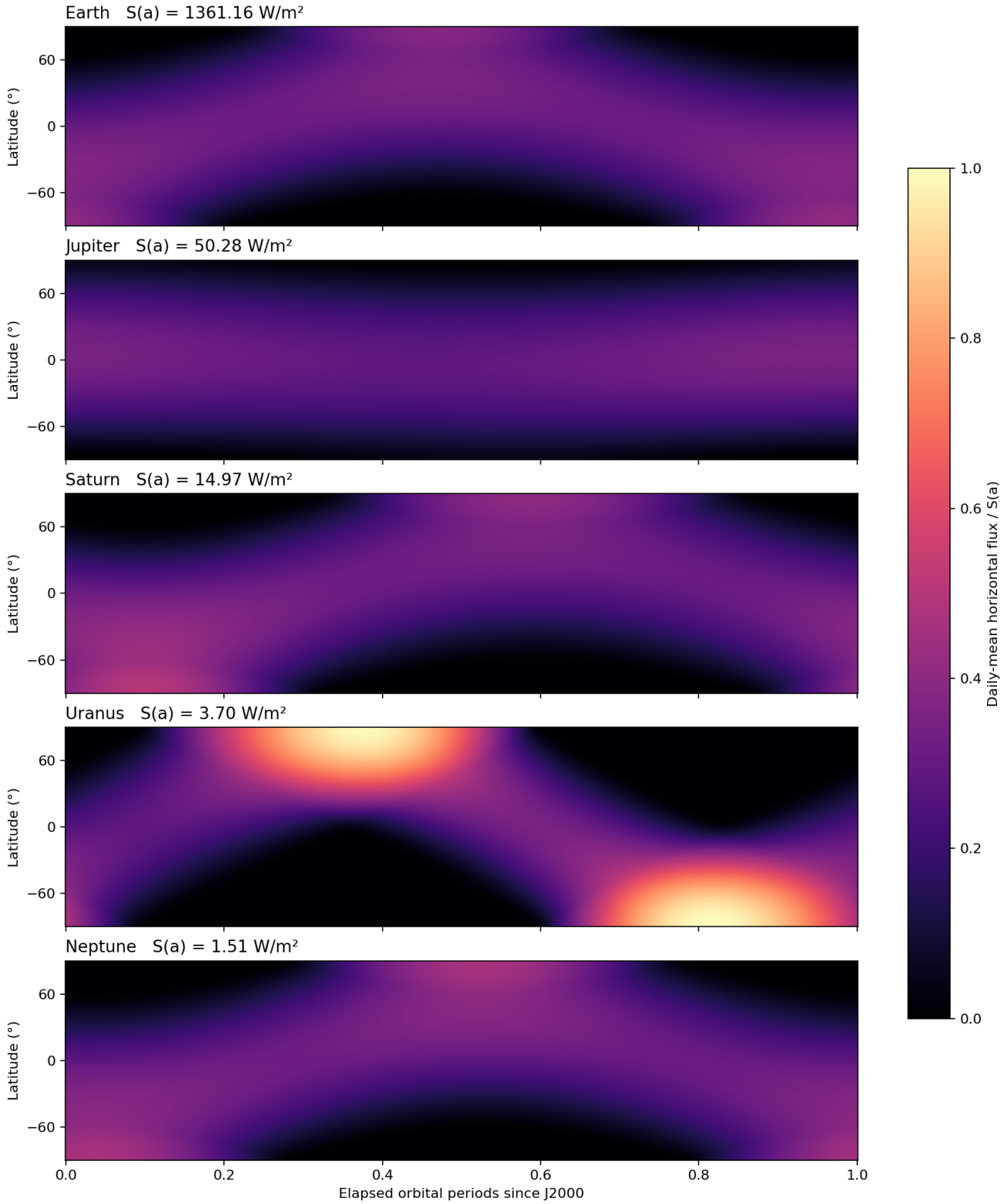
for planet_type in (Earth, Jupiter, Saturn, Uranus, Neptune):
```

```
    planet = planet_type()
    zero = torch.tensor(0.0, dtype=torch.float64)
    mu0, beam, r, lon_star, lat_star = planet.insolation(zero, zero, 0.0)
    print(planet_type.__name__, r, lat_star.rad2deg().item())

earth = Earth()
lon = torch.linspace(-torch.pi, torch.pi, 361, dtype=torch.float64)[None, :]
lat = torch.linspace(-torch.pi/2, torch.pi/2, 181, dtype=torch.float64)[:, None]
mu0, beam, *_ = earth.insolation(lon, lat, 0.0)
horizontal_flux = mu0 * beam
```



Each curve starts at J2000 and spans its own orbital period. Distance is normalized by semi-major axis. Normal irradiance shows the large interplanetary contrast; substellar latitude shows the seasonal amplitude and phase, especially the extreme seasons of Uranus.



Longitude-averaged horizontal flux, normalized by each planet's irradiance at its semi-major axis. This is the daily mean with orbital geometry held fixed during a rotation. Separate rows share one color scale, revealing polar night, polar summer, and eccentricity modulation.

The `plotting script` evaluates the actual classes for the orbit curves and uses their distances and substellar latitudes in the analytic longitude average:

$$\overline{F} = \frac{S}{\pi}[H_0 \sin\phi \sin\delta_\star + \cos\phi \cos\delta_\star \sin H_0], \qquad H_0 = \arccos[\mathrm{clip}(-\tan\phi \tan\delta_\star, -1, 1)].$$

Here $H_0 = 0$ is polar night and $H_0 = \pi$ is continuous daylight. The grid uses latitude cell centers to avoid evaluating tangents at the poles.

From the repository root, with Paddle's dependencies and Matplotlib installed:

```
MPLCONFIGDIR=/tmp/paddle-matplotlib python docs/notes/orbital/generate_figures.py
python -m pytest tests/test_orbital.py -q
```

The script uses a noninteractive backend and writes PNG and SVG versions to `figures/` next to itself; `--output-dir` selects another destination.

References

[JPL]

NASA/JPL Solar System Dynamics, Approximate Positions of the Planets, Table 1.

[PCK]

NASA/JPL NAIF, Planetary constants kernel pck00011.tpc, December 27, 2022; bodies 399, 599, 699, 799, and 899.

[IAU2015]

IAU 2015 Resolution B3, Nominal conversion constants.

[IAU2012]

IAU 2012 Resolution B2, Re-definition of the astronomical unit.

2. Gas and cloud opacity

Scope and interface

The orbit supplies the incoming beam, but not its attenuation or the thermal emission of the atmosphere. Those require the material's optical properties. `paddle/rt/opacity.py` provides two simple grey optical-property modules: `GasOpacity` combines pressure-dependent absorption from several gases, and `CloudOpacity` combines extinction and scattering from several condensates. These are prescribed parameterizations, not line-by-line gas spectroscopy or particle-size-dependent cloud scattering calculations.

Batching is over the **selected species**. Each parameter list follows the order of `species_indices`, which selects the last axis of `conc`. The module sums these contributions into one mixture. The leading output axis has length one and represents one grey spectral band, not a species batch. Use separate module instances for bands with different coefficients.

Input and output conventions

Quantity	Shape / units	Meaning
`conc`	`(ncol, nlyr, nspecies)`, mol/m³	Molar concentration of every species
`pres`	`(ncol, nlyr)`, Pa	Total pressure, shared by all gas species
`temp`	`(ncol, nlyr)`, K	Accepted for the opacity interface; unused
`molecular_weights`	List of length `nselected`, kg/mol	Molar masses of selected gases or condensates
Output	`(1, ncol, nlyr, 3)`	Extinction in 1/m, single-scattering albedo, asymmetry

In particular, molar mass must be in **kg/mol**, not g/mol. For water use approximately `0.018`, not `18`. Condensate concentration is the number of moles of condensate material per volume, not the number of cloud particles.

Each species list must have the same nonzero length. Gas `p_ref` is one positive Python float for the whole mixture, defaulting to `1.e5` Pa (1 bar). The constructor order is `species_indices, molecular_weights, kappa_ref, exponent, p_ref=1.e5`; use `p_ref=...` to override it. Floating coefficient lists are registered buffers, as are integer species indices. Move the module to the input device and floating dtype once before evaluation, for example `opacity.to(device=conc.device, dtype=conc.dtype)`. Indices remain integers. These column/layer/species calculations run on the input device; no per-cell Python loops or GPU-to-CPU transfers are needed.

Gas absorption

Let c_i be the selected molar concentration and M_i the molar mass of gas i. Its mass density and mass absorption coefficient are

$$\rho_i = c_i M_i, \qquad \kappa_i(p) = \kappa_{\mathrm{ref},i} \left(\frac{\max(p, 0)}{p_{\mathrm{ref}}} \right)^{n_i}.$$

Here `kappa_ref[i]` has units m²/kg and `exponent[i]` is dimensionless. All species share the same total pressure and the same reference pressure; this is not a partial-pressure law. The final extinction is purely absorbing:

$$\beta_{\mathrm{gas}} = \sum_i \rho_i \kappa_i(p), \qquad \omega_{\mathrm{gas}} = 0, \qquad g_{\mathrm{gas}} = 0.$$

The implementation keeps the species axis until after multiplying density by its own coefficient. For example, with two species, $\rho_1\kappa_1 + \rho_2\kappa_2$ cannot generally be replaced by a single coefficient times $\rho_1 + \rho_2$.

`pres.unsqueeze(-1)` adds the species axis to pressure. Multiplying by the one-dimensional parameter buffers broadcasts the individual pressure laws over every column and layer. The final `sum(-1)` removes only the species axis. Temperature has no explicit role; it may already affect the supplied concentrations through the atmospheric state.

Negative pressure is clamped to zero, retaining the original module's behavior. For positive exponents the opacity at zero pressure is zero; for zero exponent the pressure factor is one. Negative exponents require strictly positive pressure: zero pressure makes that power law singular. A zero concentration multiplied by an infinite coefficient can also produce NaN. The module does not regularize this singular model. Supply nonnegative finite concentrations and coefficients and physically valid pressure.

Cloud extinction and scattering

Each condensate has its own `molecular_weights`, `mass_extinction`, `single_scattering_albedo`, and `asymmetry` entry. The parameter name `mass_extinction` denotes a list of extinction coefficients in m²/kg of that condensate. For condensate j,

$$\rho_j = c_j M_j, \qquad \beta_j = \rho_j K_j, \qquad \sigma_j = \omega_j \beta_j.$$

Here K_j is mass extinction, ω_j is single-scattering albedo, and σ_j is scattering per length. Absorption per length is $(1-\omega_j)\beta_j$. Physical inputs obey $K_j \geq 0$, $0 \leq \omega_j \leq 1$, and $-1 \leq g_j \leq 1$. Pressure and temperature are unused for the prescribed cloud coefficients.

The mixture must conserve extinction and scattering, and its asymmetry must preserve the first angular moment of scattering [Mixing]. This gives

$$\beta = \sum_j \beta_j, \qquad \sigma = \sum_j \sigma_j, \qquad \omega = \frac{\sigma}{\beta}, \qquad g = \frac{\sum_j \sigma_j g_j}{\sigma}.$$

Thus albedo is **extinction-weighted**, while asymmetry is **scattering-weighted**. Neither quantity should be summed or averaged arithmetically. A purely absorbing cloud adds extinction but does not change the asymmetry of the scattering material.

When extinction is zero, the module returns `(0, 0, 0)`. When extinction is positive but scattering is zero, it returns `(beta, 0, 0)`. Albedo and asymmetry are physically irrelevant in these respective limits, so zero is an explicit numerical convention. The code replaces a zero denominator with one before dividing; the corresponding numerator is zero for physical inputs. This avoids division by zero even in branches evaluated by PyTorch, and keeps backpropagation finite at an empty cloud. It does not assign a physically unique derivative to the undefined mixture ratios at that point.

For one nonempty scattering condensate, the mixture recovers its prescribed albedo and asymmetry exactly (up to numerical roundoff).

Hand-checkable examples

Gas example: at $p = p_{\mathrm{ref}}$, let selected concentrations be $(5, 4)$ mol/m³, molar masses $(0.02, 0.01)$ kg/mol, and reference opacities $(2, 3)$ m²/kg. Then

$$\beta_{\mathrm{gas}} = 5(0.02)(2) + 4(0.01)(3) = 0.32\ \mathrm{m}^{-1}.$$

Cloud example: extinction contributions $(0.2, 0.4)$ m⁻¹, albedos $(0.5, 1.0)$, and asymmetries $(0.2, 0.8)$ give

$$\beta = 0.6\ \mathrm{m}^{-1}, \quad \sigma = 0.5\ \mathrm{m}^{-1}, \quad \omega = \frac{5}{6}, \quad g = \frac{0.1(0.2) + 0.4(0.8)}{0.5} = 0.68.$$

These values are reproduced directly by:

```
import torch
from paddle.rt.opacity import GasOpacity, CloudOpacity

conc = torch.tensor([[[4., 999., 5.]]], dtype=torch.float64)
pres = torch.tensor([[1.e5]], dtype=torch.float64)
temp = torch.tensor([[300.]], dtype=torch.float64)
gas = GasOpacity([2, 0], [0.02, 0.01], [2., 3.], [1., 2.])
gas_properties = gas(conc, pres, temp)  # [[[[0.32, 0, 0]]]]

cloud = CloudOpacity([2, 0], [0.02, 0.01], [10., 20.],
                     [0.5, 1.0], [0.2, 0.8])
condensates = torch.tensor([[[2., 999., 1.]]], dtype=torch.float64)
cloud_properties = cloud(condensates, pres, temp)
# [[[[0.6, 0.833333..., 0.68]]]]
```

Species 1 is deliberately omitted despite its large concentration. The parameters align with `[2, 0]` rather than the original concentration order. The numerical coefficients are illustrative, not calibrated material data.

Using the output in radiative transfer

Extinction is a coefficient per length, not optical thickness. For a homogeneous layer of thickness Δz,

$$\tau = \beta \Delta z.$$

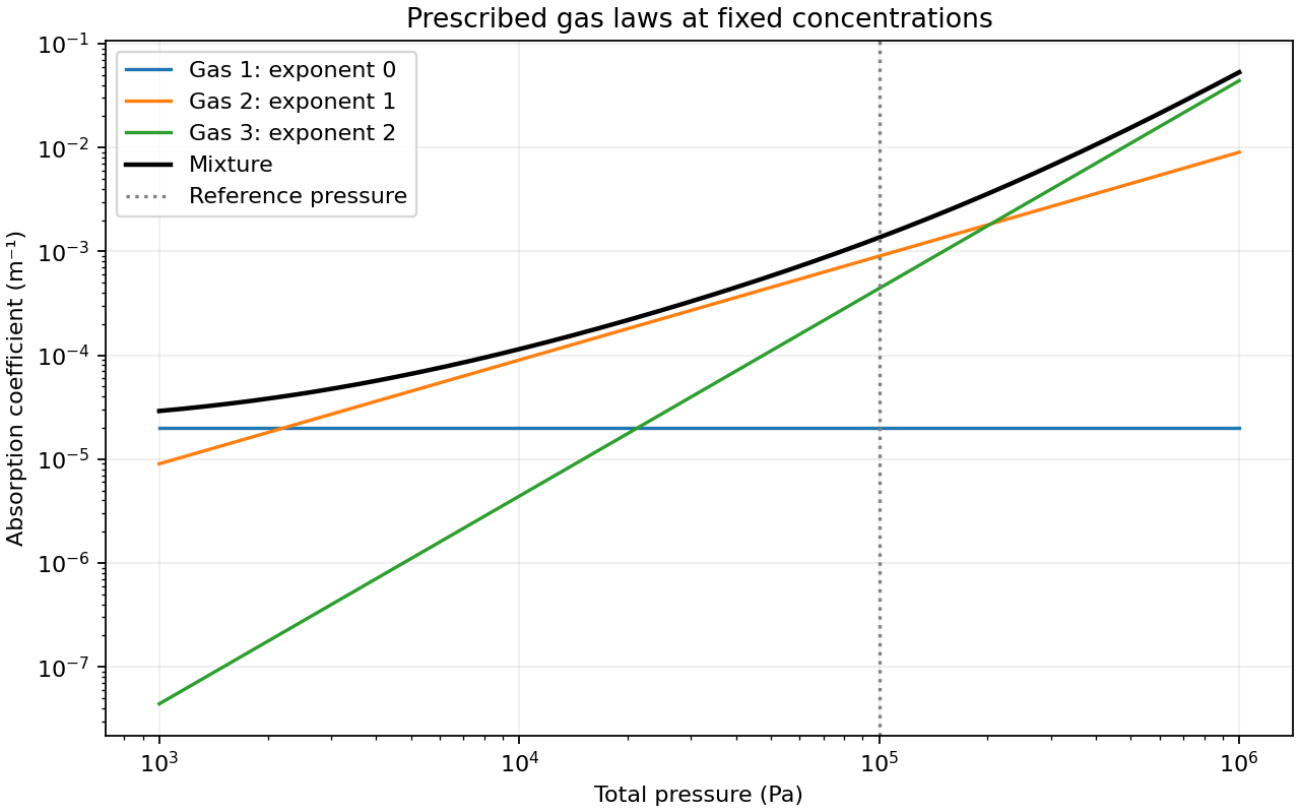
Only the first output property is multiplied by layer thickness. Albedo and asymmetry are dimensionless and remain unchanged. In the coupled workflow, PyHarp's `Radiation` performs this conversion internally, after combining the gas and cloud coefficients. Pass extinction per length from the opacity modules; do not apply `dz` a second time in `RTState`.

Likewise, gas and cloud output arrays cannot simply be added. If their extinctions are β_g, β_c and the cloud properties are ω_c, g_c, then for this purely absorbing gas model,

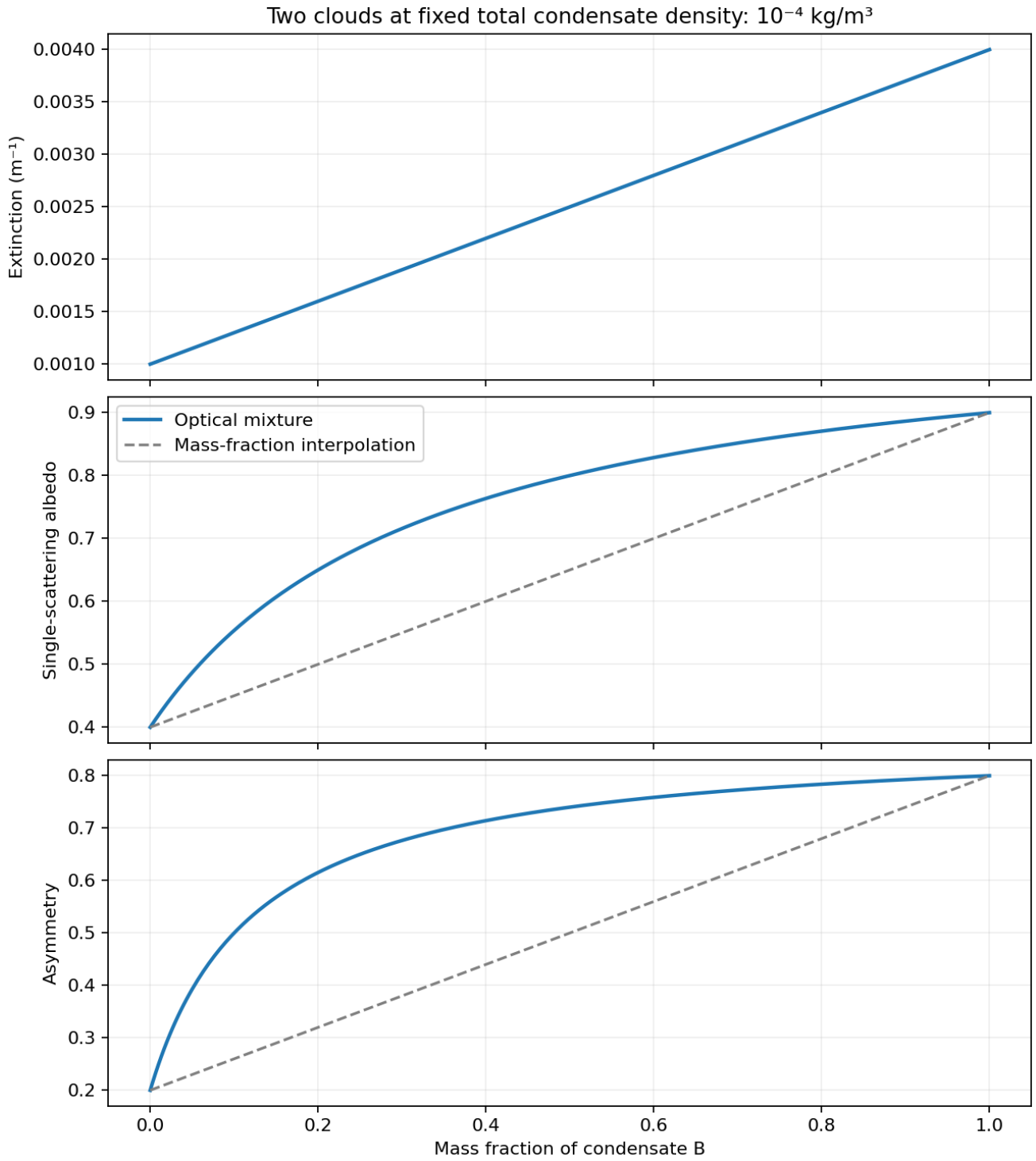
$$\beta_{\text{total}} = \beta_g + \beta_c, \qquad \omega_{\text{total}} = \frac{\beta_c \omega_c}{\beta_g + \beta_c}, \qquad g_{\text{total}} = g_c$$

when cloud scattering is nonzero. Use zero for undefined ratios, as above. PyHarp's `RadiationBand` performs this weighted combination for the opacities listed in each band. The radiation-state section connects these band definitions to the atmospheric state and completes the calculation of heating.

Examples and reproducible figures



Three prescribed gas laws at fixed concentrations. Their exponents are 0, 1, and 2; all share a reference pressure of 100 kPa. Varying pressure at fixed concentration isolates the coefficient law, rather than tracing a thermodynamically consistent atmosphere.



Two condensates at fixed total mass density, with different extinction efficiencies, albedos, and asymmetries. Increasing condensate B's mass fraction changes both weights. The dashed curves show the generally incorrect arithmetic interpolation of albedo and asymmetry by mass fraction.

The `figure-generation script` evaluates `GasOpacity` and `CloudOpacity` directly. It writes PNG and SVG versions under `figures/`; `--output-dir` chooses another directory. From the repository root, with Paddle's dependencies and Matplotlib installed:

```
MPLCONFIGDIR=/tmp/paddle-matplotlib python docs/notes/opacity/generate_figures.py
python -m pytest tests/test_opacity.py -q
python -m sphinx -W -b html docs/notes /tmp/paddle-notes-html
```

References

[Mixing]

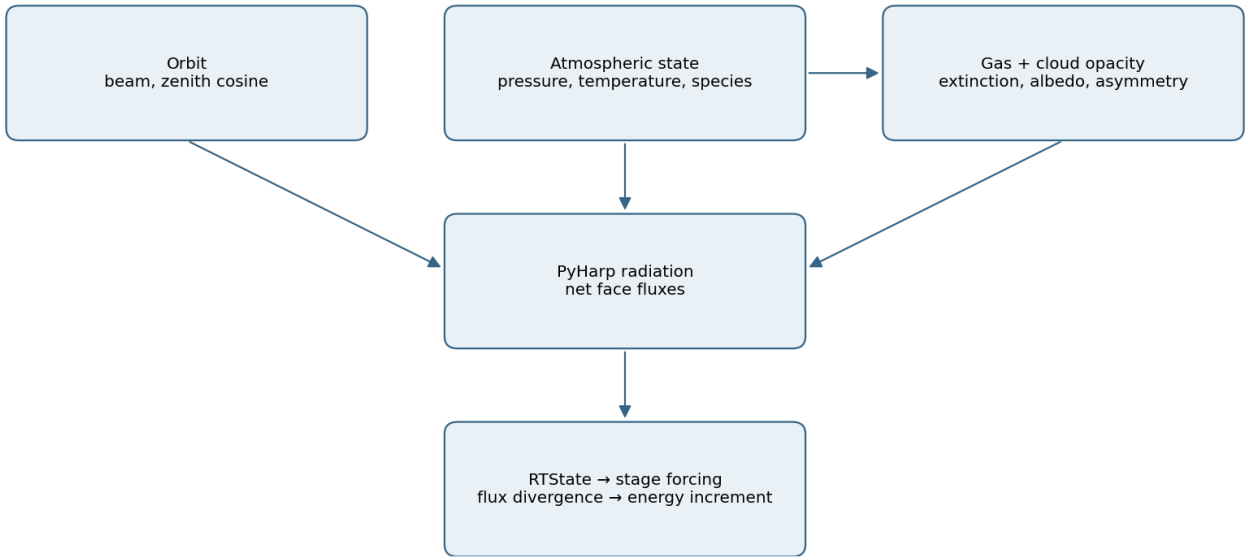
Mukherjee et al., Equivalence of internal and external mixture schemes of single scattering properties in vector radiative transfer, Applied Optics (2017). This note uses the additive-coefficient and scattering-moment definitions; its prescribed grey gas/cloud coefficients are not taken from that study.

3. Radiation state and the Uranus example

Closing the coupling

The orbit in section 1 supplies a normal stellar beam and a zenith cosine. The gas and cloud modules in section 2 supply extinction, albedo, and asymmetry. `RTState` joins those inputs to the atmospheric thermodynamic state and geometry. `RTStateCubedSphere` supplies the face-dependent illumination. PyHarp solves each vertical column and sums the spectral bands; the resulting flux divergence becomes the energy source consumed by Snapy's stage integrator.

From orbital illumination to atmospheric heating



The radiation update follows the solid arrows. Heating is held fixed between updates, while the dynamical solver uses it during each stage.

The roles are intentionally distinct:

- `OrbitalForcing.insolation` computes illumination for one scalar time.
- `GasOpacity` and `CloudOpacity` evaluate all columns and species in tensor operations on the grid device.
- `RTState.update_heating` performs a scheduled radiation solve and publishes a spatial `rt_heating` field in W/m³.
- `OrbitalForcing.forward` reads that field and returns the energy increment under Snapy's required `hydro_du` key.

From primitive variables to radiation columns

For each block, `RTState` obtains the coordinate, EOS, and mass-fraction thermodynamic modules with `block.module(...)`. It creates a `ThermoX` module using the same species definition and grid device/dtype. The species ordering is shared by thermodynamics and exported opacity modules.

Let p be pressure, T temperature, and Y_i mass fractions. The conversion sequence is

$$T = \mathrm{EOS}(W), \qquad X = \mathrm{ThermoY}(Y), \qquad c = \mathrm{ThermoX}(T, p, X).$$

In code these are `W->T`, `Y->X`, and `TPX->V`. The last operation returns molar concentrations in mol/m³ with species on the last axis, including the dry species. Its precise gas/condensate thermodynamics belong to Kintera; `RTState` does not approximate these conversions with a separate ideal-gas formula.

The hydrodynamic primitive array has shape `(nvar, nc3, nc2, nc1)`, including halo cells. Radiation uses only `kl:ku+1, jl:ju+1, il:iu+1` from the coordinate module. If the active sizes are N_3, N_2, N_1, then

$$N_{\mathrm{col}} = N_3 N_2, \qquad N_{\mathrm{lyr}} = N_1.$$

Pressure and temperature become `(ncol, nlyr)`; concentration becomes `(ncol, nlyr, nspecies)`. The radius/layer direction is last and remains ordered **bottom to top**. Flattening only the two horizontal axes preserves the same column order as the cubed-sphere longitude/latitude grid. The layer thickness array `dx1f[il:iu+1]` has units m and shape `(nlyr,)`.

For each band, the code sets PyHarp's local `ncol` and `nlyr` before constructing `Radiation`. This avoids using global YAML dimensions or including halos in the radiation solve. Relative opacity file paths are resolved against the configuration file's directory. A one-wave grey band uses weight one; multi-wave bands require explicit `weights`.

Band illumination on the cubed sphere

A process may own several blocks. `RTStateCubedSphere` uses each block's own layout rank, obtains its cube face with `loc_of(rank)`, and calls `snapy.coord.cs_ab_to_lonlat` on that block's active `x2v` and `x3v` coordinates. The resulting planetocentric latitude and east longitude are cached because the mesh geometry is fixed. Construct the state after moving the mesh to its final device/dtype; recreate it if the mesh geometry changes.

For each non-Planck band, supply one orbital model, in YAML band order. Thermal bands carry the `planck` flag; they may be interleaved with shortwave bands. The example has one shortwave and one longwave band.

With simulation time t and configured `orbit.time_offset` t_0, the orbital call uses $t + t_0$ seconds since J2000. If a restart returns the original elapsed simulation time, preserve the same t_0; do not add the restart time twice.

For a shortwave band b, the boundary dictionary contains

$$B_b = f_b B, \qquad \mu_b = \mu_0,$$

where f_b is `radiative-transfer.stellar_fractions[b]`. Fractions are nonnegative and sum to at most one. The demonstration assigns all bolometric sunlight to one grey shortwave band; its wavelength limits do not by themselves calculate the stellar spectral fraction.

The exact PyHarp keys and shapes are:

Radiation boundaries

Key	Shape	Meaning
`<band>/fbeam`	`(nwave, ncol)`	Normal stellar irradiance, W/m²
`<band>/umu0`	`(ncol,)`	Cosine of stellar zenith angle
`<band>/albedo`	`(nwave, ncol)`	Bottom-boundary reflectivity, dimensionless

Shortwave weights sum to one because each quadrature point receives the same band-integrated grey beam. This convention avoids duplicating stellar energy when several points are used. It is not a general stellar-spectrum quadrature.

On the night side, `fbeam` is exactly zero and `umu0` is set to one. That positive dummy cosine prevents the Toon kernel from evaluating divisions by zero; it does not introduce sunlight because the beam is zero. Thermal radiation remains active on both day and night sides. Bottom albedo is distinct from an atmospheric particle's single-scattering albedo. It is applied to every band, including the thermal band.

Flux divergence and energy conservation

PyHarp combines opacity sources with the weighting derived in section 2, converts extinction to optical thickness, solves the bands, and returns `net_flux` in W/m². Net flux is **upward minus downward**, on layer faces ordered bottom to top. The returned shape is `(ncol, nlyr+1)`. For thermal emission, the current PyHarp implementation interpolates the provided layer temperatures to levels; `RTState` supplies `temp` rather than its own independent interpolation.

Let $A_{i\pm1/2}$ be radial face areas and V_i cell volume. The volumetric heating is the negative finite-volume flux divergence:

$$q_i = -\frac{A_{i+1/2}F_{i+1/2} - A_{i-1/2}F_{i-1/2}}{V_i}.$$

Positive q_i adds energy; cooling is negative. The units are $(\mathrm{m}^2\, mW/m^2)/\mathrm{m}^3 = \mathrm{W}/\mathrm{m}^3$. Using the actual spherical cell geometry gives the column budget

$$\sum_i q_i V_i = A_{\mathrm{bottom}}F_{\mathrm{bottom}} - A_{\mathrm{top}}F_{\mathrm{top}}.$$

This telescoping identity is checked by the example. It verifies conservative coupling, not the accuracy of the prescribed grey opacities or plane-parallel column transfer. Horizontal radiative transport, limb curvature, ring shadows, and changes in beam direction across a column are not represented here. Only the flux divergence uses spherical areas/volumes; no additional PyHarp spherical-rescaling inputs are supplied.

`rt_heating` has the spatial shape `(nc3, nc2, nc1)`, **without** a hydrodynamic variable axis. The active cells receive q_i; halos remain zero. A separate `net_flux` snapshot is kept for diagnostics. Nonfinite heating raises an error before replacing the previously cached result.

Scheduling and stage integration

Radiation is expensive relative to applying a cached source. The first call always solves, including when the simulation begins at a nonzero restart time. After a successful solve at t_r,

$$t_{\mathrm{next}} = t_r + \Delta t_{\mathrm{RT}}.$$

Calls in $[t_r, t_{\mathrm{next}})$ reattach the cached field to the variable dictionary. At the deadline a new solve is due. A time rewind to before t_r also forces recomputation. `update_dt=0` means solve on every call. Each block owns its own cache and boundary configuration; no class-level mutable lists are shared. On restart, create new states so the first radiation call rebuilds heating rather than depending on a checkpointed cache.

At the beginning of each accepted hydrodynamic step, the example updates all radiation states, then calls `mesh.forward` for every integrator stage. The saved `OrbitalForcing` module returns

$$(\Delta U)_j = \begin{cases} q\Delta t, & j = \mathrm{kIPR}, \\ 0, & j \neq \mathrm{kIPR}. \end{cases}$$

It reads `hydro_u` for the increment's shape and returns `{"hydro_du": increment}`, which is Snapy's stage-callback contract. The energy index is copied from `snapy.kIPR` into an instance integer for TorchScript. `stage` is accepted but the integrator owns stage weighting. The callback does not perform another radiation solve or modify its inputs. Do not apply the same increment again outside the integrator.

Holding heating fixed is a temporal approximation. Choose `update_dt` to resolve the evolution of both the atmosphere and its illumination. The example clips the hydro step to the next radiation deadline and also caps it at one second for a short demonstration. It aborts on a failed step; a production integration needs its own step-retry policy and timestep study.

Running the Uranus example

`paddle/uranus.yaml` contains seven thermodynamic species, a 12-layer radial grid, and an 8-by-8 active grid on each cube face. Six local blocks cover all faces in one process. Three halos support the configured reconstruction. The original large-grid setup is reduced to make this example practical on CPU as well as GPU.

`example_py_scripts/uranus_radiation.py` performs the following sequence:

1. Read the YAML and derive molar masses from Kintera's species definition.
2. Export each gas/cloud opacity as TorchScript, using indices into the **full** concentration vector, not the smaller YAML opacity species list. The JIT opacity receives the full vector in the current PyHarp interface.
3. Write a resolved YAML with absolute opacity paths into the output directory.
4. Construct the mesh and orbital stage callback, initialize a dry-adiabatic profile, and seed trace methane and hydrogen-sulfide cloud mass fractions.

5. Construct one radiation state per block; update heating and integrate the requested short run.

6. Solve radiation again at the final state, check the column energy budgets, and save tensors for plotting in `radiation.pt`.

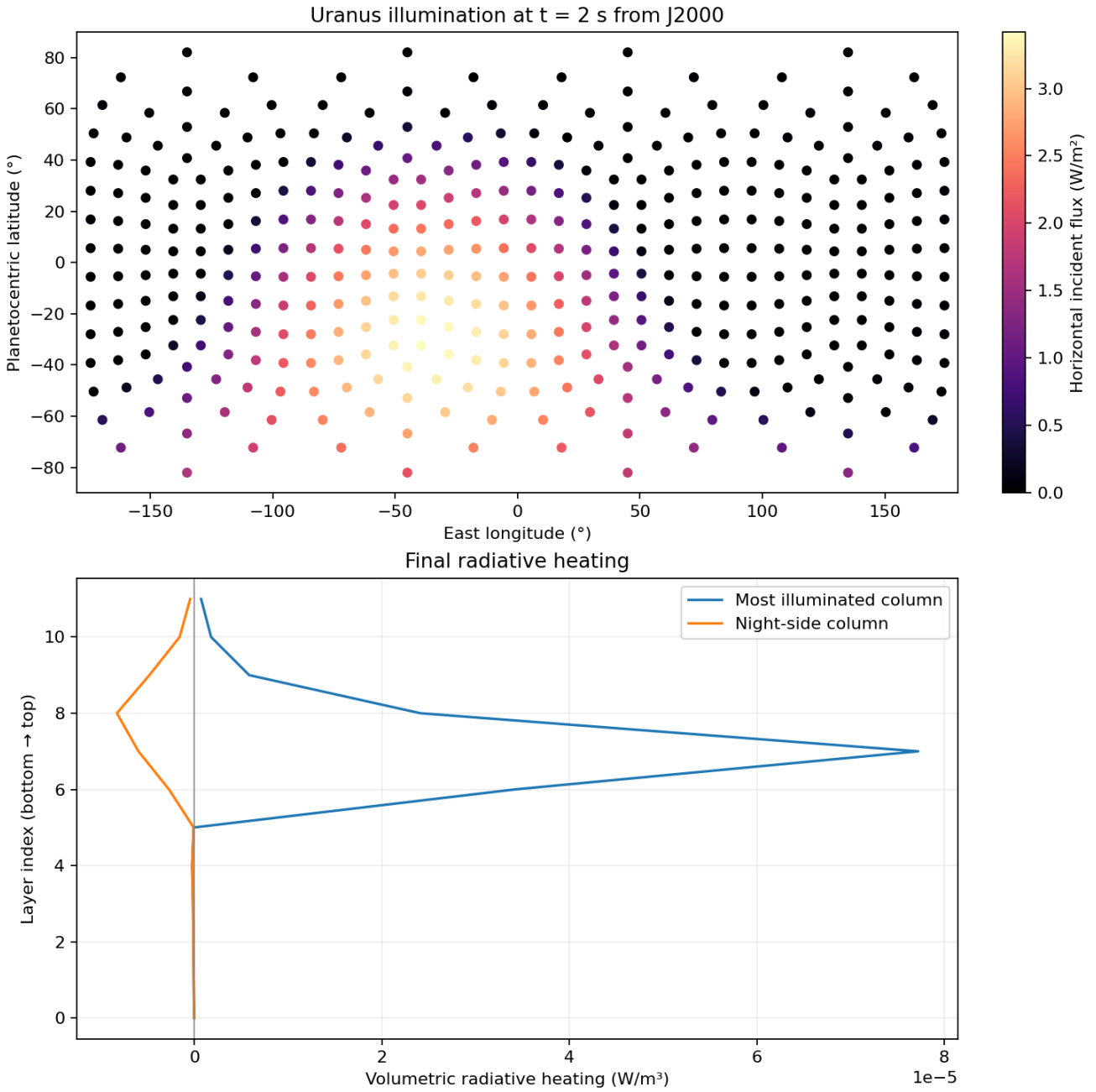
The clouds are prescribed small initial mass fractions, not the result of a cloud-microphysics spin-up. The example does not call the kinetic reaction integrator. Gravity, Coriolis forcing and bottom heating remain configured in the YAML. The Coriolis spin has the same negative sign as the Uranus preset, using the IAU north-pole convention. The grey radiation coefficients are illustrative. The radiation budget check concerns the radiative source alone, not those additional hydrodynamic energy terms.

The current PyHarp JIT-opacity wrapper loads its inner script on the device stored in the file; moving the enclosing `Radiation` object is insufficient to relocate that inner script. The preparation step therefore exports separate artifacts for the selected device. Use different output directories for CPU and GPU runs. All calculations in this example use float64.

With Snapy, Kintera and PyHarp installed:

```
OMP_NUM_THREADS=1 python example_py_scripts/uranus_radiation.py \
    --device cpu --output-dir /tmp/uranus-cpu --steps 2
OMP_NUM_THREADS=1 python example_py_scripts/uranus_radiation.py \
    --device cuda --output-dir /tmp/uranus-cuda --steps 2
```

PyHarp is available through Paddle's `radiation` optional dependency group. The default YAML is resolved relative to the script, so the example can also be invoked by absolute path from another working directory. Its generated YAML, models, and diagnostics are written only to `--output-dir`.



A two-step CPU run. Points show the physical horizontal incident flux, `fbeam * umu0`; the dummy night cosine leaves these points dark. The lower panel shows final volumetric heating profiles sampled from the most and least illuminated columns. Heating includes thermal cooling.

Validation and reproduction

The focused tests check active-cell slicing, the sign and area/volume factors in heating, cache deadlines, time rewinds, empty/restarted caches, failure handling, band mapping, and night-side boundaries. Integration tests execute the real six-face example on CPU and CUDA, check distinct face coordinates, finite fields, and column energy closure. Orbital and opacity tests also cover TorchScript and both float32/float64 optical modules.

```
OMP_NUM_THREADS=1 python -m pytest tests/test_rt_state.py \
    tests/test_uranus_radiation.py tests/test_opacity.py tests/test_orbital.py -q
MPLCONFIGDIR=/tmp/paddle-matplotlib python docs/notes/radiation/generate_figures.py \
    --diagnostics /tmp/uranus-cpu/radiation.pt
python -m sphinx -W -b html docs/notes /tmp/paddle-notes-html
```

The `figure script` writes PNG and SVG figures next to this section. Sections 1 and 2 retain their own reproducible figure scripts. The combined PDF follows the same order: orbital boundary forcing, material opacity, and radiation-state coupling.

To build the combined PDF, install Sphinx, WeasyPrint and tinyhtml5 in your Python environment, and MathJax 3 for Node.js. Then run from the repository root:

```
npm install --prefix /tmp/paddle-pdf-node mathjax-full@3
python -m sphinx -W -b singlehtml -D master_doc=radiation/index \
    -D exclude_patterns=index.rst,_build docs/notes /tmp/paddle-radiation-html
NODE_PATH=/tmp/paddle-pdf-node/node_modules \
    python docs/notes/radiation/build_pdf.py \
    --html /tmp/paddle-radiation-html/radiation/index.html
```

The `PDF builder` uses the accompanying `MathJax renderer` to embed equations as vector images before printing. The output is `docs/notes/radiation/radiation.pdf`.